Document number: P3608R0
Audience: EWG

Ville Voutilainen
Jonathan Wakely
Gabriel Dos Reis
2025-01-28

Contracts and profiles: what can we reasonably ship in C++26

Abstract

This paper proposes that we postpone novel ideas to either white papers that we ship before C++29, or to C++29.

Furthermore, this paper proposes that we ship exactly three things related to these problem domains in C++26:

1. the general profiles framework, P3589.
2. the standard library hardening, P3471.
3. a concrete profile that switches on the standard library hardening, and makes the violations of hardened preconditions just terminate the program, without any additional flexibility for C++26. Implementation vendors are, however, encouraged not to close the door for other violation handling strategies as extensions or as standardized strategies in the future.

Ship everything else when it's mature, including the additional flexibility of a violation of a hardened precondition invoking a user-defined hook, so as to prevent unconditional termination.

If we think we absolutely have to ship some other profile, like the proposed Types profile, we can ship it as a profile that must be enabled as experimental (e.g. as suggested in P3447).

Rationale

Contracts have unaddressed design issues and disagreements (see P3573), especially in the realm of how contracts work on virtual functions. We have neither a good agreement on that nor actual deployment experience on reasonable-scale use of contracts on virtual functions.

We have various outstanding issues with the current profiles proposal, especially the parts where it adds expression-level language checks for things like uses of operator[], and how that interacts with libraries that for various reasons do not need nor want such checks.

We have a very unclear picture on how contracts and profiles should interact and interoperate.

And all this with our noses pressed against a deadline. We are rushing things in, without employing prudent engineering practices, most importantly the quality assurance ones, quite plainly meaning especially testing.

In contrast to that, the standard library hardening is existing practice, and comes with very positive field experience reports. Two out of three of our major library vendors already ship it, and the third one is working on it actively. The missing piece in the picture for the wider C++ programmer audience is how to enable that checking.

The debates on what we absolutely need right now

Various members of our group will happily tell you that we are in dire need to ship various more-featured solutions immediately, in C++26, or else the whole user community faces unmitigable perils.

We should avoid the temptation of pushing unfinished facilities in because we feel an urgency.

I must also point out that if such impending doom is upon us, we need to deal with it effectively, with solutions and responses that actually work, aren't guesswork, and are field-tested. Otherwise there's a very good chance that hastily adopted solutions end up making the situation worse, because we will end up over-advertising the capabilities of solutions, and thereby shipping something much worse and less functional than smaller incremental steps.

Summary

Ship the stable and mature existing practice. Don't ship wild guesses. Ship what's in our current pipeline for profiles and contracts once we've had the chance to work on them further, to resolve various issues, to improve consensus, and to make all those facilities work together as a coherent whole.